

GUIA BASE DEL ARQUITECTO DE SOLUCIONES EN EL SECTOR FINANCIERO

Roberto Carlos Catuta

INDICE

1. Introducción: El Papel Crítico del Arquitecto Financiero

- 1.1 Definición del rol en el sector bancario
- 1.2 Responsabilidades clave
- 1.3 Ejemplo práctico: Implementación de pagos QR

2. Dominio Técnico Profundo

2.1 Arquitecturas Clave

2.1.1 Microservicios Avanzados

- Concepto y características
- Diagrama de arquitectura bancaria
- Caso de uso: Sistema de transferencias

2.1.2 Event-Driven Architecture (EDA)

- Componentes (Productores, Consumidores, Brokers)
- Ejemplo con Apache Kafka
- Flujo completo de una transacción

2.2 Manejo de Datos

2.2.1 Bases de Datos Financieras

- SQL (Oracle, SQL Server)
- NoSQL (MongoDB, Redis)
- Tabla comparativa de usos

2.2.2 Data Warehousing y Analytics

- Soluciones en la nube (Azure Synapse)
- Procesamiento de big data en banca

3. Seguridad y Cumplimiento Normativo

3.1 Estándares Regulatorios

3.1.1 PCI-DSS (Protección de Datos de Tarjetas)

- Requisitos técnicos detallados
- Ejemplo de implementación

3.1.2 PSD2 (Open Banking)

- Impacto en arquitecturas
- Diseño de APIs seguras

3.1.3 Basel III

- Requerimientos de capital
- Sistemas de cálculo automático

3.2 Arquitecturas Seguras

3.2.1 Zero Trust Architecture

- Principios fundamentales
- Implementación paso a paso

3.2.2 Hardware Security Modules (HSM)

- Casos de uso en transacciones
- Integración con sistemas legacy

4. Resiliencia y Alta Disponibilidad

4.1 Patrones Esenciales

4.1.1 Circuit Breaker

- Explicación técnica
- Ejemplo en código (Python/Java)

4.1.2 SAGA Pattern

- Flujo completo de transacciones
- Mecanismos de compensación

4.1.3 CQRS (Command Query Responsibility Segregation)

- Aplicación en reportes financieros

4.2 Estrategias de Recuperación

4.2.1 Definiciones clave (RTO, RPO)

4.2.2 Planes de Disaster Recovery

4.2.3 Backup automatizados

5. Habilidades de Negocio y Gestión

5.1 Comunicación Efectiva

5.1.1 Lenguaje técnico vs. lenguaje de negocio

5.1.2 Ejemplos reales de diálogos

5.2 Gestión de Proyectos Financieros

5.2.1 Metodologías (Agile, ITIL)

5.2.2 Herramientas (Jira, MS Project)

5.2.3 Manejo de presupuestos

6. Caso de Estudio Completo

6.1 Modernización de Core Bancario

6.1.1 Situación inicial (problemas del monolito)

6.1.2 Solución implementada

6.1.3 Resultados cuantificables

6.2 Diagramas de Arquitectura

6.2.1 Arquitectura antes y después

6.2.2 Flujo de datos

7. Tendencias Futuras

7.1 Computación Cuántica y su impacto

7.2 Inteligencia Artificial en fraudes

7.3 Blockchain para identidad digital

8. Recursos para Aprender

8.1 Cursos recomendados (con enlaces)

8.2 Libros especializados

8.3 Certificaciones clave

9. Las 15 Cosas que debe saber

A. Conocimientos Técnicos

1. Microservicios
2. Sistemas de mensajería (Kafka)
3. APIs (REST/gRPC)
4. Bases de datos financieras
5. Automatización (DevOps)

B. Seguridad y Cumplimiento

6. PCI-DSS
7. PSD2
8. Basel III
9. OAuth 2.0/JWT
10. HSM

C. Habilidades de Negocio

11. Core bancario
12. Comunicación con no técnicos
13. Gestión de proyectos

D. Resiliencia Operacional

14. Circuit Breaker

15. Estrategias de backup

10. Conclusión

10.1 Resumen de competencias

10.2 Perfil ideal del arquitecto financiero

1. Introducción: El Papel Crítico del Arquitecto Financiero

En el mundo de la banca y las finanzas, el arquitecto de soluciones es como el **director de orquesta** de todos los sistemas tecnológicos. Mientras que los desarrolladores programan y los operadores mantienen los servidores, el arquitecto:

- **Diseña** cómo todos los componentes trabajarán juntos
- **Asegura** que cumplan con leyes estrictas
- **Previene** fallos que podrían costar millones
- **Innovar** sin poner en riesgo la estabilidad

Ejemplo real:

Cuando tu banco lanza una nueva función (como pagos con QR), el arquitecto decidió:

1. Cómo se conectará con los sistemas antiguos
2. Cómo protegerá los datos
3. Cómo manejará 10,000 transacciones por segundo

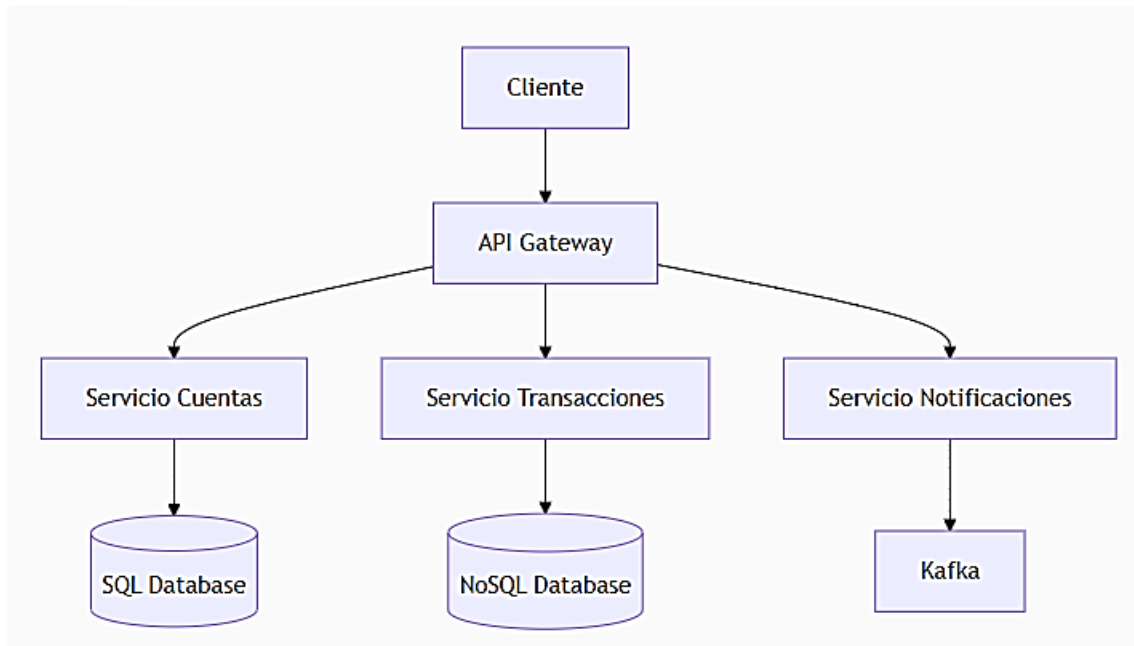
2. Dominio Técnico Profundo

2.1 Arquitecturas Clave Explicadas

Microservicios Avanzados

- **Qué es:** Dividir un gran sistema en pequeños servicios independientes
- **Detalle técnico:**
 - Cada servicio tiene su propia base de datos
 - Se comunican mediante APIs o mensajes
 - Escalan individualmente

Ejemplo bancario:



Event-Driven Architecture (EDA)

- **Para qué sirve:** Procesar eventos en tiempo real (transferencias, alertas de fraude)
- **Componentes:**
 - **Productores:** Generan eventos (ej: "Se realizó un pago")
 - **Consumidores:** Procesan eventos (ej: "Actualizar saldo")
 - **Broker:** Distribuye eventos (Kafka, RabbitMQ)
- **Caso de uso:**

"Cuando un cliente transfiere \$1000:

1. Servicio Transacciones publica evento
2. Servicio Cuentas actualiza saldos
3. Servicio Anti-Fraude analiza patrones"

2.2 Manejo de Datos Financieros

Bases de Datos Especializadas

Bases de Datos Especializadas

Tipo	Para qué sirve	Ejemplo Bancario
SQL (Oracle)	Transacciones ACID	Movimientos de cuentas
NoSQL (MongoDB)	Datos flexibles	Perfiles de clientes
In-Memory (Redis)	Ultra-rápido	Caché de saldos

Data Warehousing

- **Qué es:** Almacenes para reportes y analytics
- **Herramientas:**
 - Azure Synapse
 - Amazon Redshift
- **Ejemplo:**

"Cálculo diario de riesgo crediticio para 5M clientes"

3. Seguridad y Cumplimiento Normativo

3.1 Estándares Clave Desglosados

PCI-DSS (Payment Card Industry)

- **Objetivo:** Proteger datos de tarjetas
- **Requisitos clave:**
 1. Encriptar datos en tránsito y reposo
 2. Restringir acceso solo a personal autorizado
 3. Testear sistemas regularmente

PSD2 (Open Banking)

- **Qué permite:**
 - Bancos deben compartir datos (con consentimiento)
 - Terceros pueden iniciar pagos
- **Impacto técnico:**
 - APIs seguras con OAuth2.0
 - Mecanismos de consentimiento

3.2 Arquitecturas Seguras

Zero Trust Architecture

- **Principio:** "Nunca confíes, siempre verifica"
- **Implementación:**
 - Autenticación multifactor en cada acceso
 - Microsegmentación de redes

HSM (Hardware Security Module)

- **Función:** Generar y proteger claves criptográficas
- **Uso típico:**
 - Firmar transacciones
 - Encriptar datos sensible

4. Resiliencia y Alta Disponibilidad

4.1 Patrones Clave

Circuit Breaker

- **Problema:** Un servicio caído puede colapsar todo
- **Solución:**
 - *# Pseudocódigo*
 - **if** servicio_fallando:
 - desviar_tráfico()
 - usar_backup()

SAGA Pattern

- **Para qué:** Transacciones distribuidas
- **Ejemplo Bancario:**

"Transferencia entre bancos:

1. Banco A debita
2. Si Banco B falla, Banco A reembolsa"

4.2 Estrategias de Recuperación

Concepto	Definición	Ejemplo Bancario
RTO	Tiempo máximo de caída	1 hora para pagos
RPO	Máxima pérdida de datos	5 segundos de transacciones

5. Habilidades de Negocio y Gestión

5.1 Comunicación Efectiva

- **Técnico → Negocio:**

"Kafka permite procesar 10K transacciones/segundo, reduciendo costos en 30%"

- **Negocio → Técnico:**

"Necesitamos cumplir la nueva ley de fraude en 6 meses"

5.2 Gestión de Proyectos

- **Metodologías:**
 - Agile para desarrollo
 - ITIL para operaciones
- **Herramientas:**
 - Jira para seguimiento
 - Visio para diagramas

6. Caso de Estudio Completo

Proyecto: Modernización de Core Bancario

6.1 Situación Inicial

- Sistema monolítico de 20 años
- 8 horas de mantenimiento mensual
- Incapaz de manejar pagos instantáneos

6.2 Solución Arquitectónica

1. **Estrategia:** Strangler Pattern

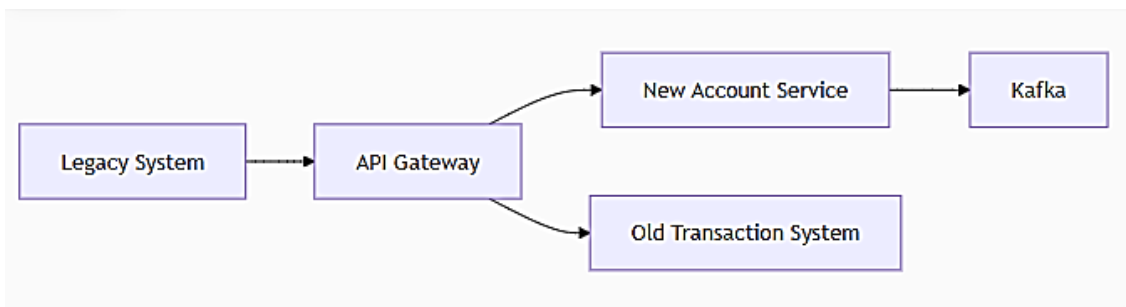
- Reemplazar gradualmente funciones

2. Tecnologías:

- Frontend: Angular
- Backend: Java Spring Boot
- Integración: Apache Kafka

3. Resultados:

- 99.99% disponibilidad
- 300ms en transacciones



7. Tendencias Futuras

- **Quantum Computing:** Nuevos retos de seguridad
- **AI/ML:**
 - Detección de fraude en tiempo real
 - Chatbots para servicio al cliente
- **Blockchain:**
 - Pagos internacionales
 - Identidad digital

8. Recursos para Aprender

Tipo	Recurso	Enlace
Curso	Azure for Financial Services	Microsoft Learn
Libro	Building Microservices	O'Reilly
Certificación	AWS Certified Solutions Architect	AWS Training

9. 15 Cosas que Debe Saber

A. Cosas Técnicas Importantes

1. Microservicios

- ¿Qué es? En lugar de una app gigante (como un elefante), se divide en "apps pequeñas" (como hormigas) que trabajan juntas.
- Ejemplo: En un banco, una "app" para transferencias, otra para préstamos, etc.

2. Kafka / Azure Event Hubs

- ¿Qué es? Un "WhatsApp" entre sistemas. Cuando pasa algo (ej: un pago), avisa a otros servicios.
- Ejemplo: Si transfieres dinero, avisa al servicio de notificaciones para que te llegue un SMS.

3. APIs (REST/gRPC)

- ¿Qué es? "Lenguaje común" para que los sistemas se entiendan.
- Ejemplo: Cuando usas una app bancaria, esta habla con el servidor del banco mediante APIs.

4. Bases de Datos SQL y NoSQL

- ¿Qué es?
 - SQL: Como Excel, pero para guardar datos de clientes (ej: Oracle, SQL Server).

- NoSQL: Como Google Drive, para guardar cosas menos organizadas (ej: fotos de perfiles).

5. DevOps y Automatización

- ¿Qué es? Usar robots (scripts) para desplegar sistemas sin errores.
- Ejemplo: En vez de instalar manualmente una app, un bot lo hace en segundos.

B. Seguridad y Leyes (¡Esto es Clave en Bancos!)

6. PCI-DSS

- ¿Qué es? Reglas para proteger tarjetas de crédito. Si el banco no las cumple, le multan.
- Ejemplo: Los números de tarjetas nunca se guardan sin encriptar.

7. PSD2

- ¿Qué es? Ley europea que obliga a los bancos a compartir datos (con tu permiso).
- Ejemplo: Apps como Revolut usan esta ley para acceder a tu cuenta bancaria (solo si tú lo autorizas).

8. Basel III

- ¿Qué es? Reglas para que los bancos no quiebren. Deben guardar dinero extra por si hay crisis.
- Ejemplo: El arquitecto diseña sistemas que calculan automáticamente estos "ahorros".

9. OAuth 2.0 / JWT

- ¿Qué es? Credenciales digitales para acceder a sistemas.
- Ejemplo: Cuando inicias sesión con tu huella, usan OAuth.

10. HSM (Hardware Security Module)

- ¿Qué es? Una "caja fuerte" electrónica para guardar claves secretas.
- Ejemplo: Donde los bancos guardan las claves para firmar transferencias.

C. Habilidades de Negocio

11. Entender el "Core Bancario"

- ¿Qué es? Saber cómo funcionan préstamos, transferencias, tarjetas, etc.
- Ejemplo: Si el banco lanza una nueva tarjeta, el arquitecto debe diseñar el sistema que la soporte.

12. Hablar con No Técnicos

- ¿Qué es? Explicar a abogados o gerentes "por qué necesitamos Kafka" sin usar palabras raras.

13. Gestionar Proyectos

- ¿Qué es? Organizar equipos, plazos y presupuestos.
- Ejemplo: Decidir si contratar AWS o Azure para ahorrar dinero.

D. Cosas para Evitar Desastres

14. Circuit Breaker

- ¿Qué es? Como un "fusible" que evita que un sistema colapse.
- Ejemplo: Si el servicio de pagos falla, el Circuit Breaker redirige a un backup.

15. Backups y Recuperación

- ¿Qué es? Tener copias de seguridad por si hackean el banco.
- Ejemplo: Si se cae el sistema, recuperarlo en 1 hora (no en 1 semana).

10. Conclusión Final

Un arquitecto financiero exitoso combina:

- 40% conocimiento técnico profundo
- 30% entendimiento regulatorio
- 20% habilidades de comunicación
- 10% visión innovadora